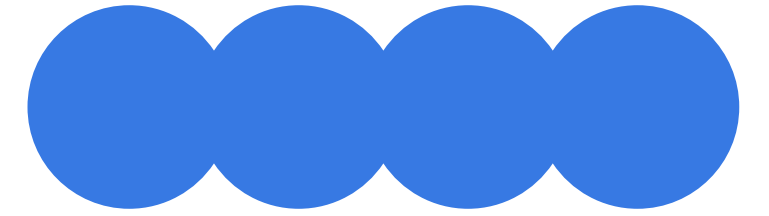


NYC Building Insights

cosmic-spaghetti.

Mery and Najihah

Project Overview



What did we build?

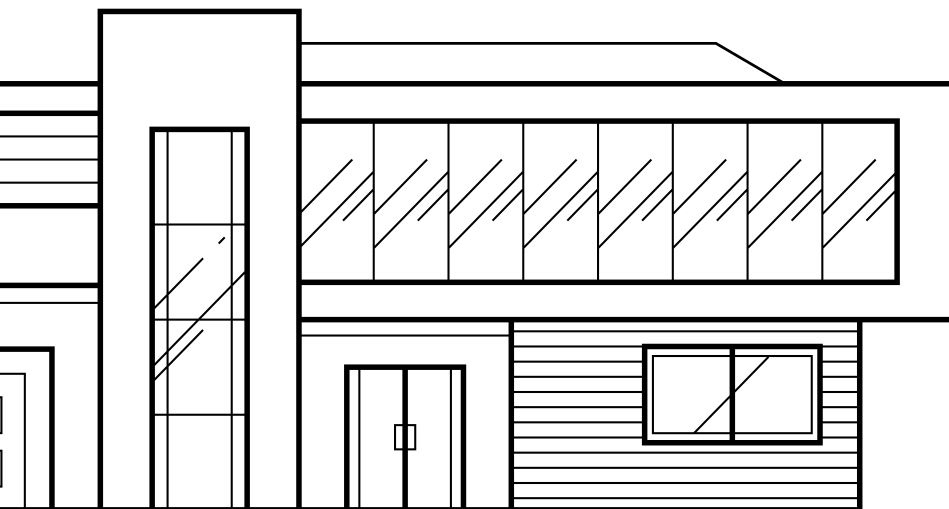
An interactive Streamlit dashboard exploring NYC building permits, evictions and complaints across all five boroughs.

Why it matters?

NYC is currently experiencing its worst housing affordability crisis due to a severe supply-demand imbalance. The city also has a building safety crisis due to aging infrastructure, construction challenges, and widespread scaffolding across the city.

How?

Pulled live data from NYC Open Data APIs
↓
BigQuery
↓
Streamlit



Evolution of our Proposal



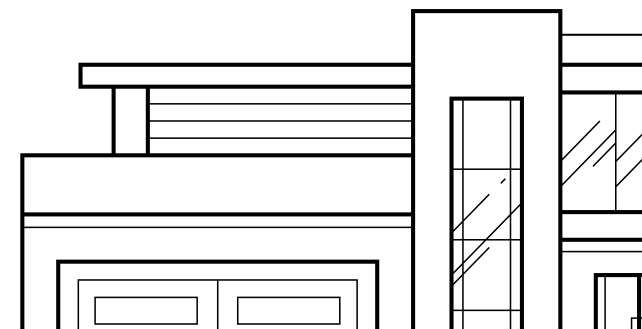
Initial Proposal

- Initially focused on housing affordability and evictions only
- Core dataset was NYC Open Data's Evictions datasets and ACS Census Income
- Visuals: choropleth map of eviction rates by borough, basic line chart for periodic trends
- Wanted to explore a policy question, e.g. correlation between income vs eviction



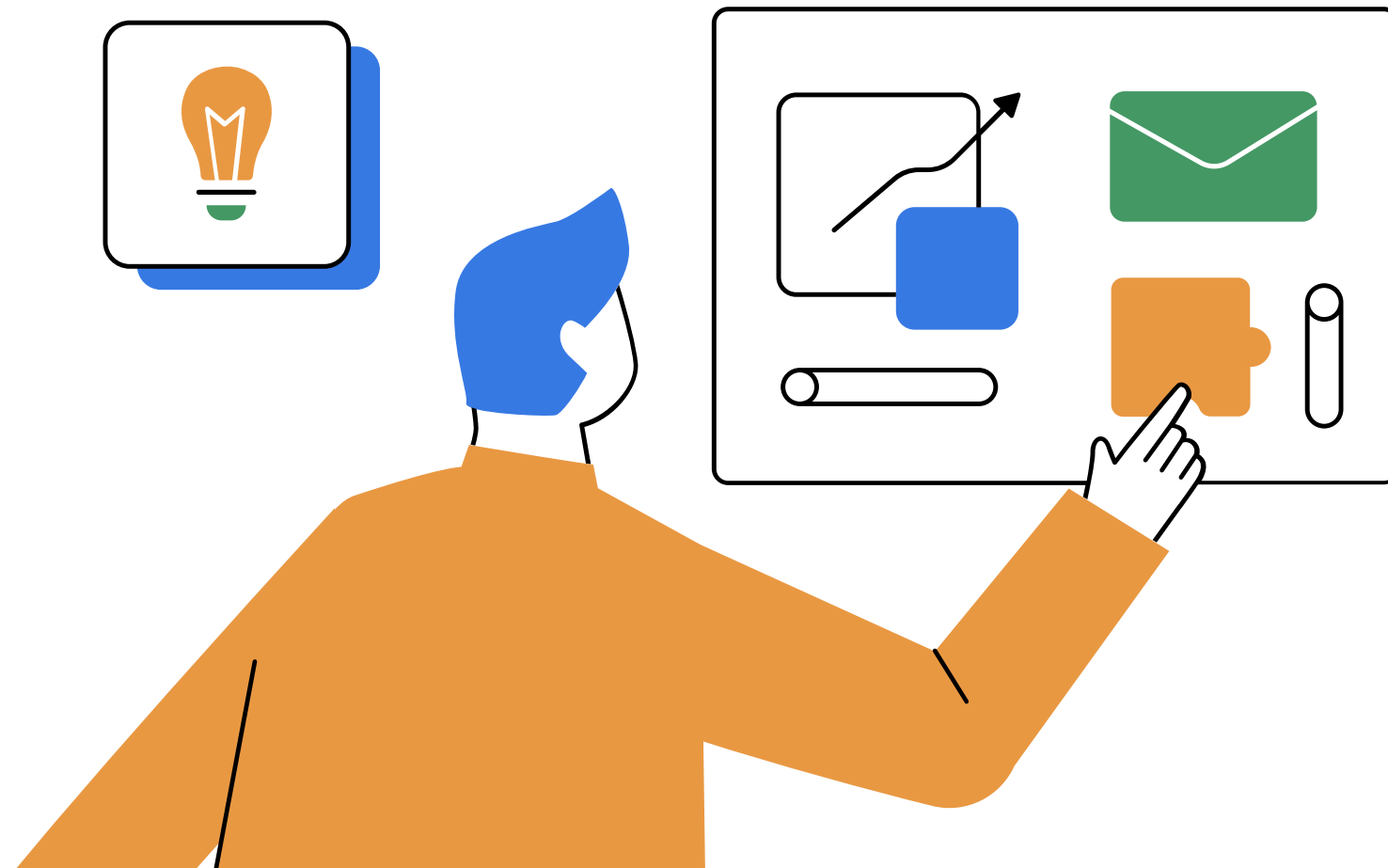
Final Scope

- Expanded to a dashboard where we can look at building construction, evictions, and complaints
- Added other datasets, including DOB Permits, Building Footprints, and DOB Complaints to generate understanding surrounding housing and building in NYC
- BigQuery replaced direct API calls
- Daily refresh via GitHub Actions
- Improved README and setup docs



Code Organization

to improve efficiency, performance and peace of mind



Functions

Created functions to generate visualizations across different datasets (e.g. filtering, time series, etc.)

Data pipeline

Created scripts to load datasets from the API to BQ

Pages

Create pages for separate goals: Building Overview, Evictions Stats and Complaints Stats

Automatic updates

Scheduled workflows to ensure data is regularly updated via GitHub Actions

Testing

Created pytest to ensure that code functions well before pushing to GitHub

Code quality and formatting

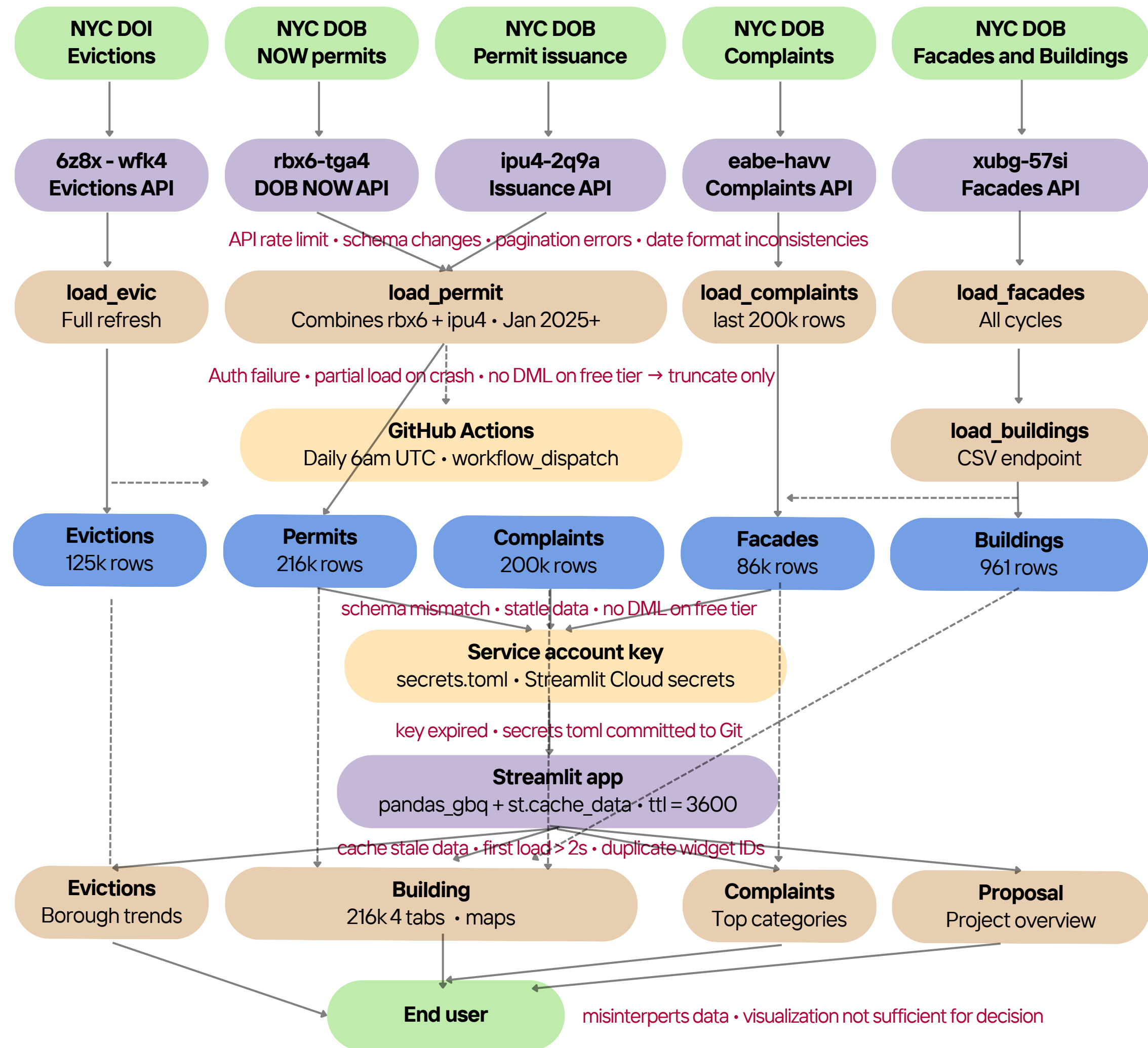
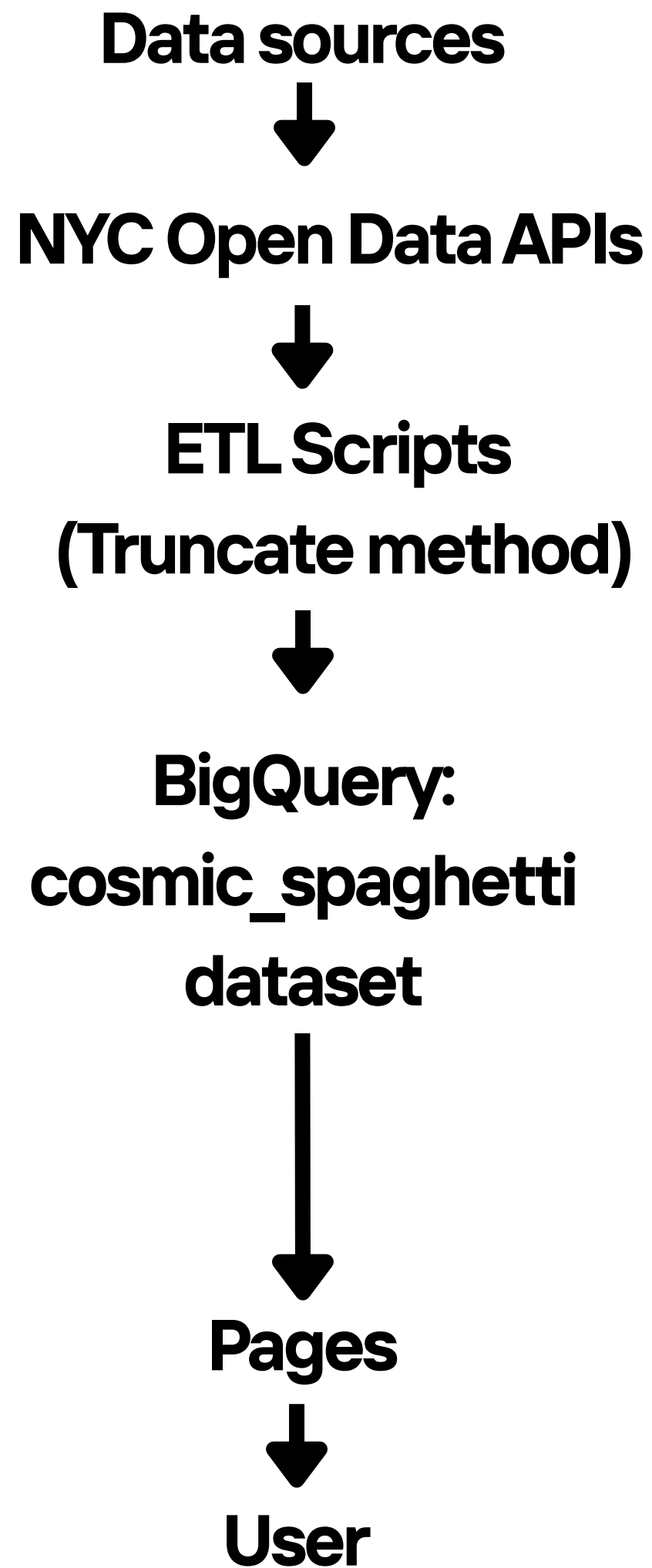
Used Ruff for linting and formatting to maintain clean, consistent code.

Used Pandera for data validation.

File Structure

📁 .github/workflows
📁 .vscode
📁 functions
📁 pages
📁 tests
📄 .gitignore
📄 README.md
📄 Setting up BigQuery with pandas and Strea...
📄 complaint_categories.py
📄 cosmic-spaghetti-notebook-housing.ipynb
📄 load_buildings_to_bq.py
📄 load_complaints_to_bq.py
📄 load_evic_to_bq.py
📄 load_facades_to_bq.py
📄 load_permit_to_bq.py
📄 pyproject.toml
📄 requirements.txt
📄 streamlit_app.py

1. dashboard pages placed in the pages folder
2. functions for visualization, dashboard theme, API pagination and data validation.
3. README file, making it easier to duplicate.



ETL 1/4

combining two datasets into one table

```
# Dataset 1 Follow link \(cmd + click\) permits (newer filings, work permits: GC, PL, ME)
URL_NOW = "https://data.cityofnewyork.us/resource/rbx6-tga4.json"

# Dataset 2 – DOB Permit Issuance (older system, job types: NB, DM, A1, A2, A3)
# Also has LATITUDE / LONGITUDE
URL_ISSUANCE = "https://data.cityofnewyork.us/resource/ipu4-2q9a.json"

limit = 50000
```

ETL 2/4

sharing a pagination function

```
def fetch_paginated(url: str, params: dict) -> list:
    """Generic paginated fetch from NYC Open Data API."""
    all_records = []
    offset = 0
    session = requests.Session()

    while True:
        params["$limit"] = limit
        params["$offset"] = offset
        response = session.get(url, params=params)
        response.raise_for_status()
        chunk = response.json()

        if not chunk:
            break
        all_records.extend(chunk)
        offset += limit
        print(f"  Fetched {offset} rows so far...")
        if len(chunk) < limit:
```


ETL 3/4

we handle datasets differently

```
def fetch_now_permits() -> pd.DataFrame:
    print("\nFetching DOB NOW permits (rbx6-tga4) from Jan 2025...")
    params = {
        "$where": f"issued_date >= '{START_DATE}'", # use constant
    }
    records = fetch_paginated(URL_NOW, params)
    df = pd.DataFrame(records)

    if df.empty:
        return pd.DataFrame()
```

```
def fetch_issuance_permits() -> pd.DataFrame:
    print("\nFetching DOB Permit Issuance (ipu4-2q9a) - NB jobs only...")
    params = {
        "$where": "job_type = 'NB'", # filter by job type only
    }
    records = fetch_paginated(URL_ISSUANCE, params)
    df = pd.DataFrame(records)

    if df.empty:
        return pd.DataFrame()
```

ETL 2/4

We standardize and combine

```
# Main
if __name__ == "__main__":
    # fetch both datasets
    df_now = fetch_now_permits()
    df_issuance = fetch_issuance_permits()

    # combine
    df_combined = pd.concat([df_now, df_issuance], ignore_index=True)

    # standardize
    df_combined["permit_date"] = pd.to_datetime(
        df_combined["permit_date"], format="mixed", errors="coerce"
    )
    df_combined["borough"] = df_combined["borough"].str.strip().str.upper()
    df_combined["permit_type"] = df_combined["permit_type"].str.strip().str.upper()
    df_combined["latitude"] = pd.to_numeric(df_combined["latitude"], errors="coerce")
    df_combined["longitude"] = pd.to_numeric(df_combined["longitude"], errors="coerce")
```

updating the data-truncate

```
name: Update Big Query

on:
  schedule:
    - cron: '0 6 * * *' → Runs at 06:00 # Runs every day at 6:00 AM UTC
  workflow_dispatch: # Allows manual triggering

jobs:
  update:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.13'
```

```
return df

def upload_to_bq(df: pd.DataFrame) -> None:
    """Technique: TRUNCATE (replace)
    Combined permits from DOB NOW and DOB Permit Issuance.
    Starting from January 2025.
    Full refresh on each run."""
    pandas_gbq.to_gbq(
        df,
        table_id,
        project_id=project_id,
        if_exists="replace",
    )
    print(f"\n Uploaded {len(df):,} rows to {table_id}")
```

```
1     name: Update Big Query
2
3     on:
4         schedule:
5             - cron: '0 6 * * *' # Runs every day at 6:00 AM UTC
6         workflow_dispatch: # Allows manual triggering
7
```

Talking about performance

```
@st.cache_data(ttl=3600, show_spinner=False)
def load_other_permits() -> pd.DataFrame:
    one_year_ago = (datetime.now() - timedelta(days=365)).strftime("%Y-%m-%d")
    query = f"""
    SELECT borough, permit_date, permit_type, permit_type_desc,
           status, latitude, longitude, source
    FROM `{PROJECT_ID}.{DATASET}.permits`
    WHERE permit_type != 'NB'
    AND permit_date >= '{one_year_ago}'
    AND borough IS NOT NULL
    LIMIT 50000
    """
    df = pandas_gbq.read_gbq(
        query,
        project_id=PROJECT_ID,
        credentials=get_credentials(),
        progress_bar_type=None,
        dtypes={"borough": "str", "permit_type": "str", "status": "str"},
    )
    df["permit_date"] = pd.to_datetime(df["permit_date"], errors="coerce")
    df["latitude"] = pd.to_numeric(df["latitude"], errors="coerce")
    df["longitude"] = pd.to_numeric(df["longitude"], errors="coerce")
    return df
```


Interesting Bug

```
def fetch_buildings() -> list:
    """Fetch building footprints via CSV endpoint – avoids geometry API issues."""
    all_records = []
    offset = 0
    session = requests.Session()

    # use CSV endpoint instead of JSON – works on geospatial datasets
    csv_url = "https://data.cityofnewyork.us/resource/5zhs-2jue.csv"

    print("Fetching NYC building footprints (CSV endpoint)...")
    while True:
        params = {
            "$limit": limit,
            "$offset": offset,
        }
        response = session.get(csv_url, params=params)
        response.raise_for_status()

        df_chunk = pd.read_csv(io.StringIO(response.text))
        if df_chunk.empty:
            break
```

Dashboard Features

New Building Jobs (2008–2020)

Source: DOB Permit Issuance (ipu4-2q9a) — job_type = NB · 2008–2020 only

▼

What is a New Building Job?

A **New Building (NB)** job is filed with the NYC Department of Buildings (DOB) when a property owner or developer intends to construct a brand new building on a lot.

Job Type	Description
NB	New Building — construct a new structure from scratch
A1	Major Alteration — change in use, egress, or occupancy

▼

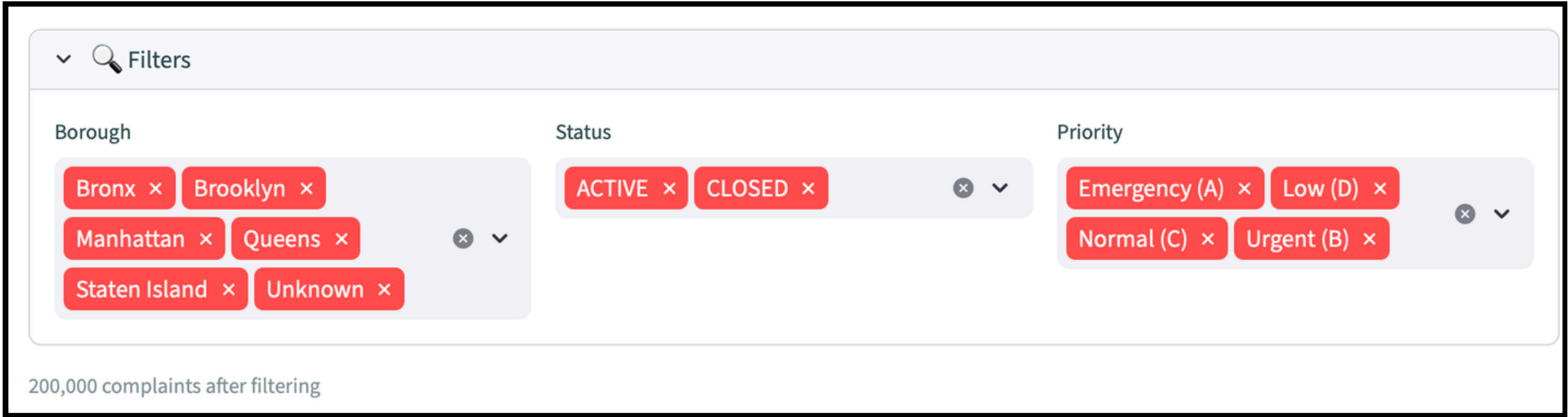
How is response time calculated?

Response time is calculated as the number of days between when a complaint was filed (`date_entered`) and when it was resolved (`disposition_date`).

Priority	DOB Target Response Time
A — Emergency	Same day inspection
B — Urgent	Within 5 business days

Collapsible info panel, helping non-policymakers understand the dashboard better

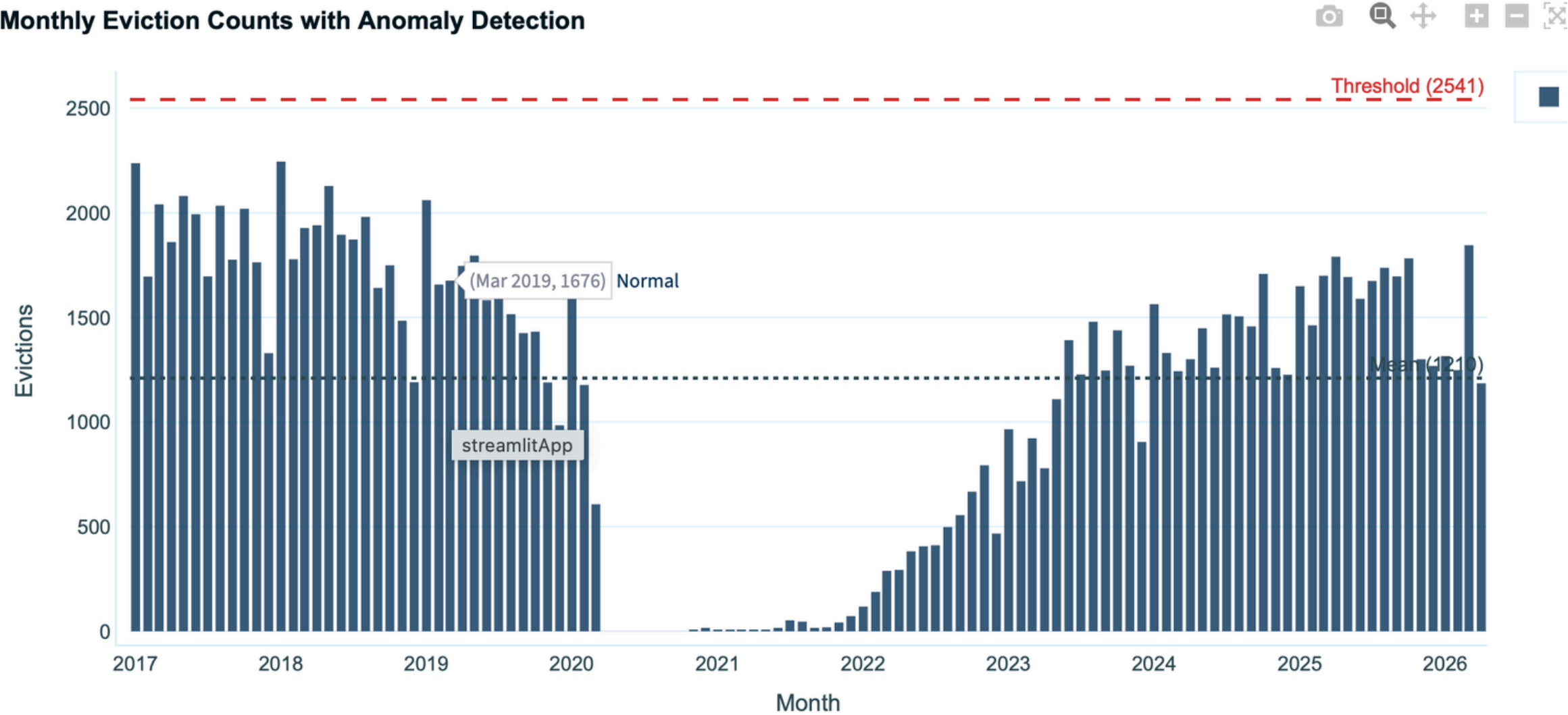
Dashboard Features



Collapsible filter, keeping the dashboard neat

Dashboard Features

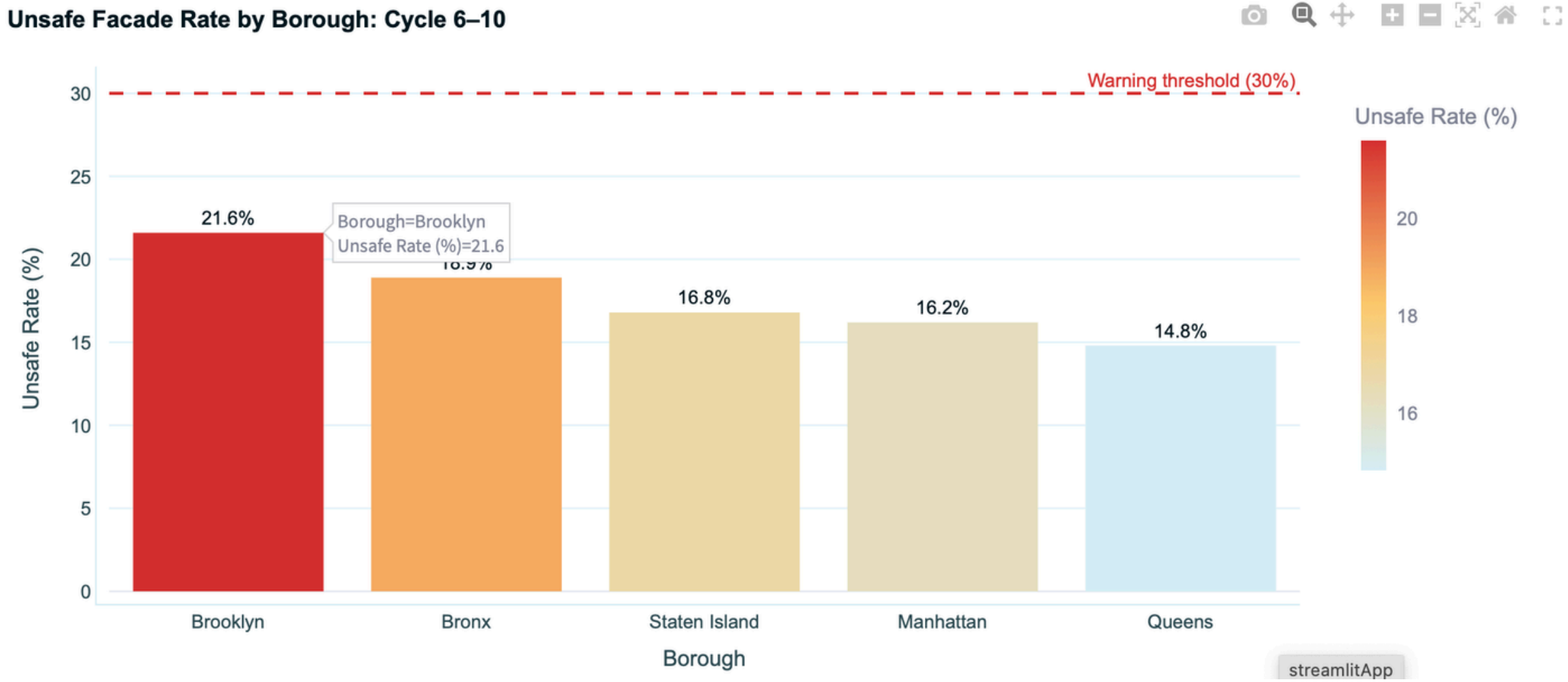
Monthly Eviction Counts with Anomaly Detection



Anomaly Detection, calculated from mean and standard deviation of the dataset

Dashboard Features

Unsafe Rate by Borough

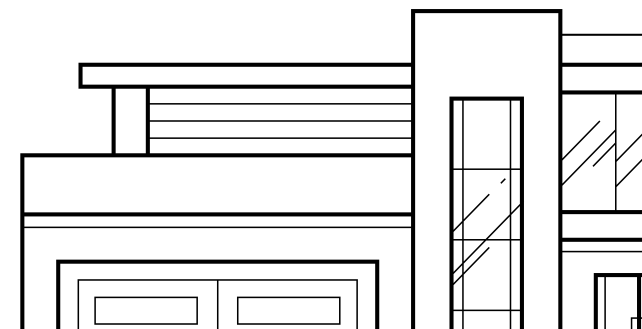


Warning Threshold: An alert appears when Unsafe Rate exceeds Threshold



Takeaways: Technical Stuff (and non technical stuff)

1. The Agile Manifesto is not just theory, being agile is what got us to the finish line.
2. Data Engineering requires data understanding, not all datasets prepared the same, even when sources are the same.
3. SQL + Python and @st.cache are powerful to enhance performance
4. Better color palette <https://colors.co/93b7be-f1fffa-d5c7bc-a79090-785964-454545>

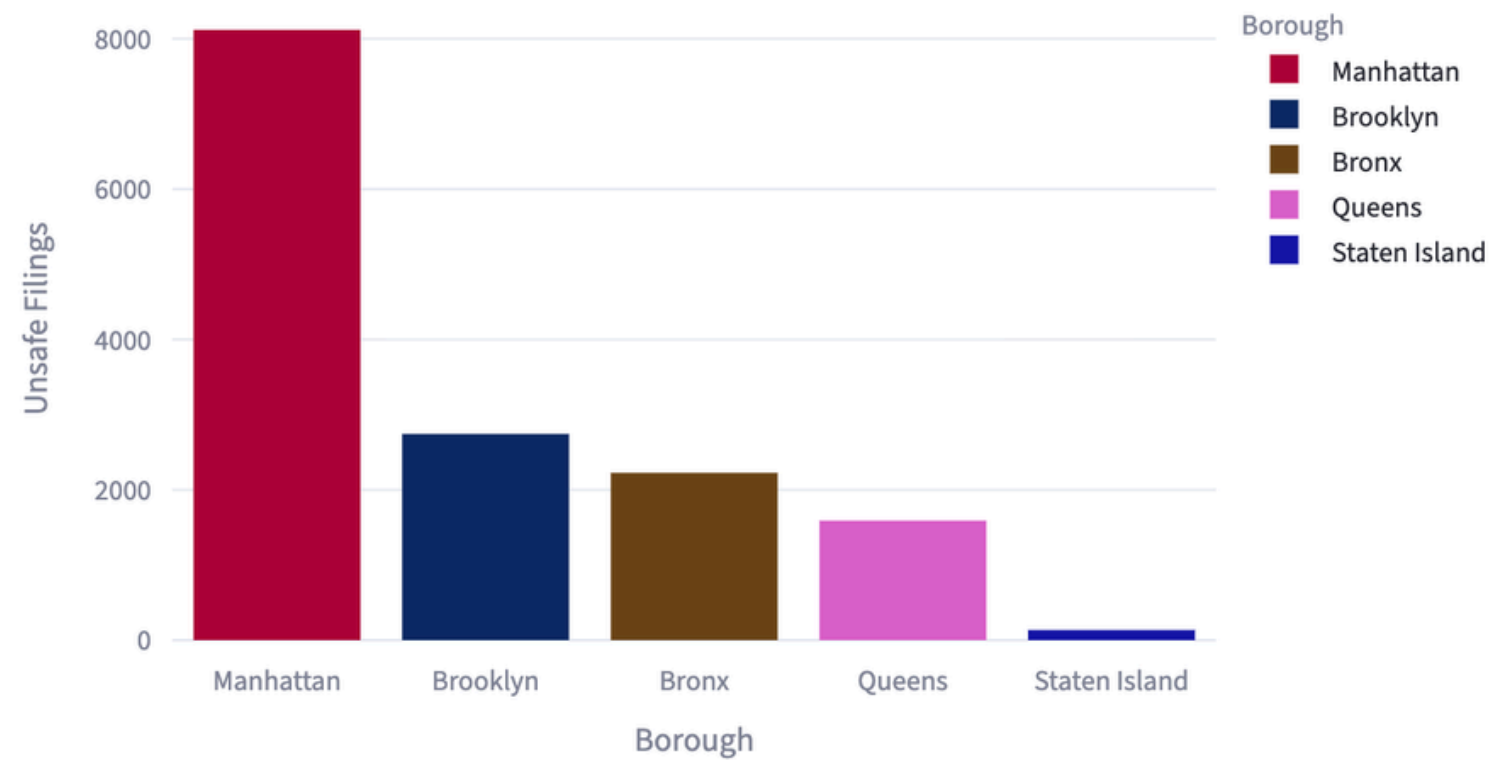


Takeaways



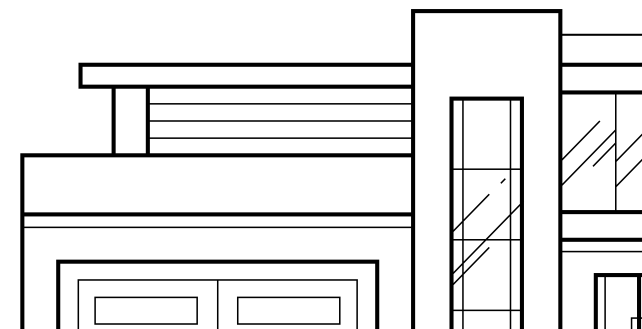
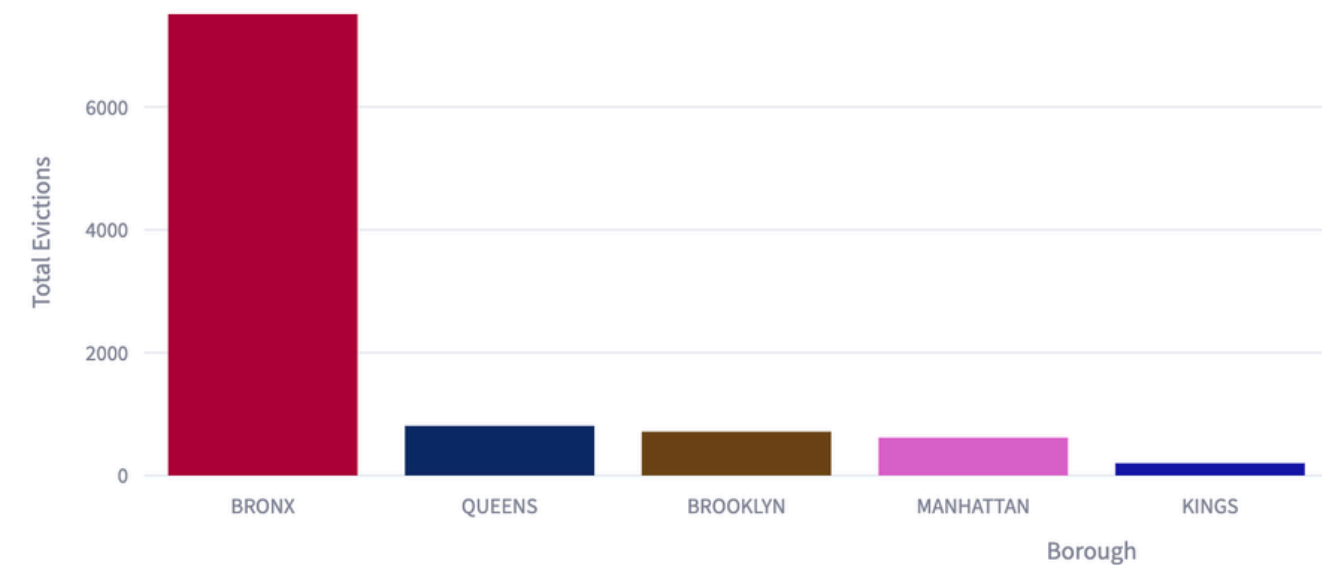
1. Manhattan has unsafe facades

Unsafe Facade Filings by Borough



2. Bronx has the highest number of evictions.

NYC Evictions by Borough

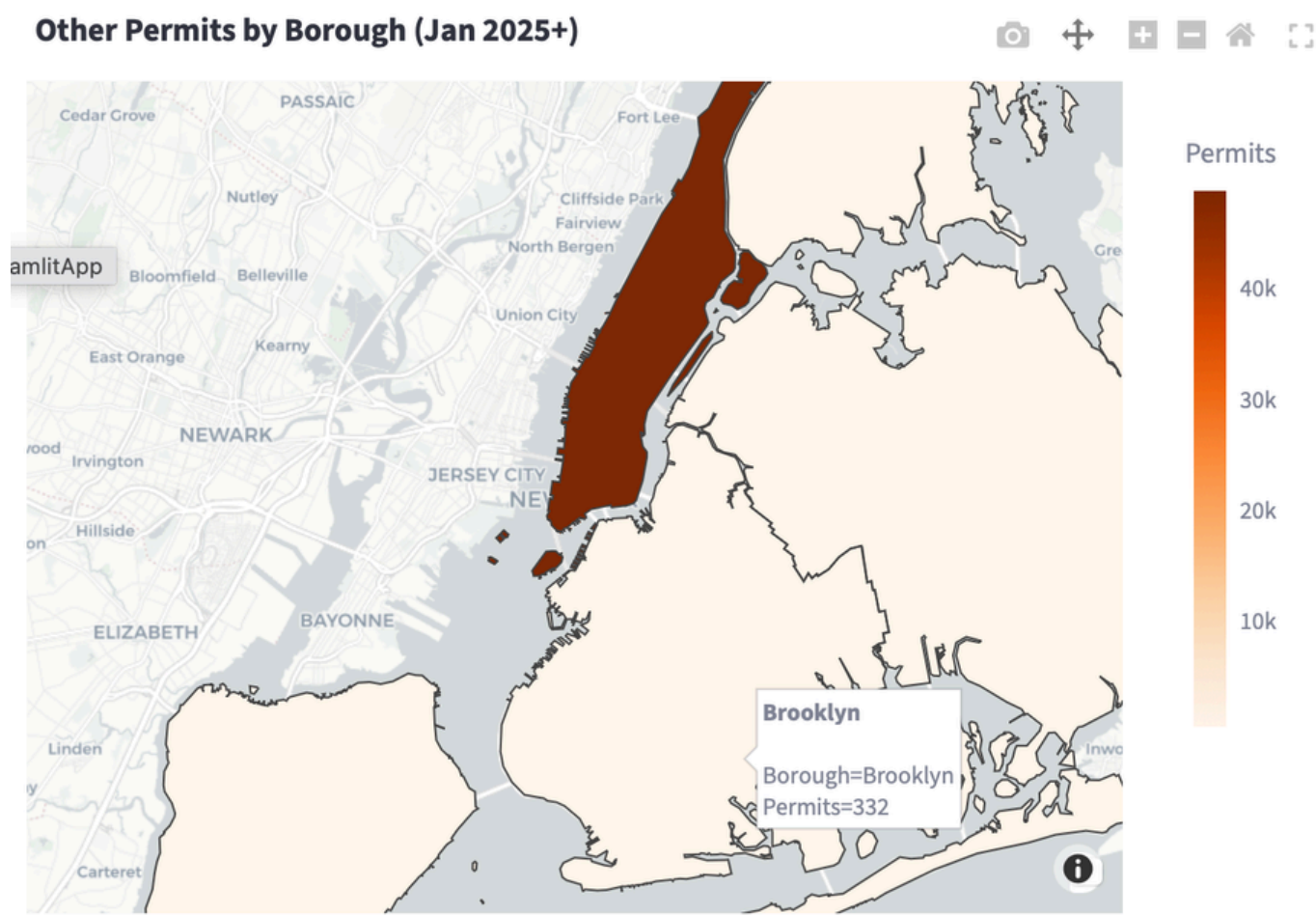


Takeaways

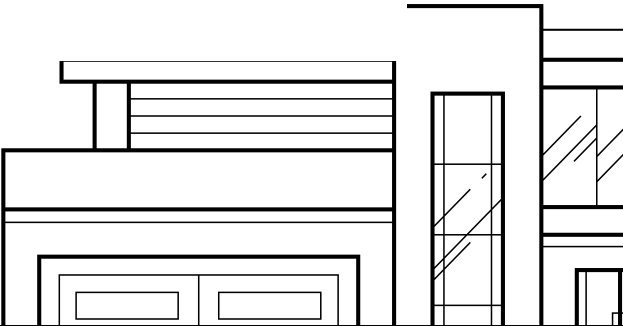
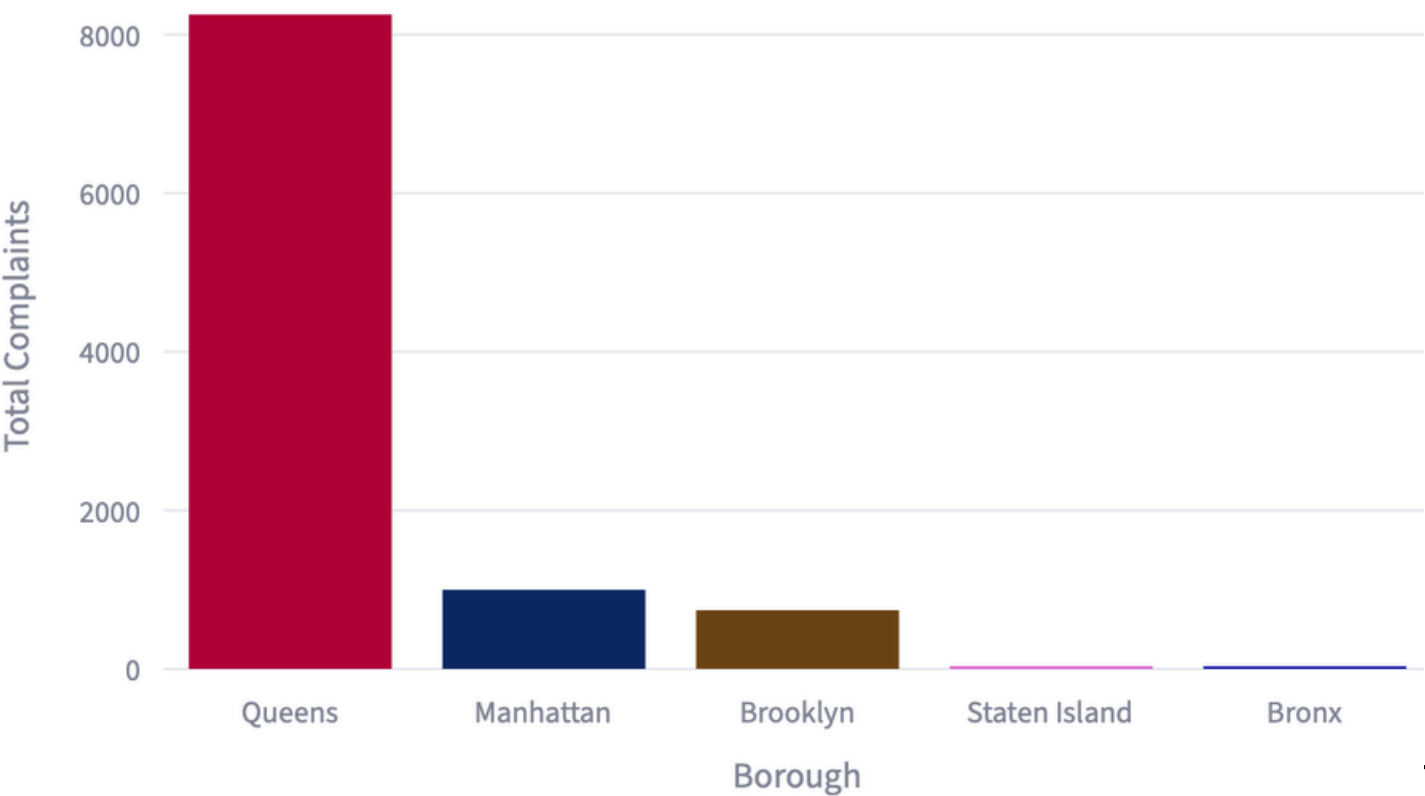


3. But most of building renovations happens in Manhattan

4. Queens has the most building complaints



NYC Building Complaints by Borough



Who is this useful for?

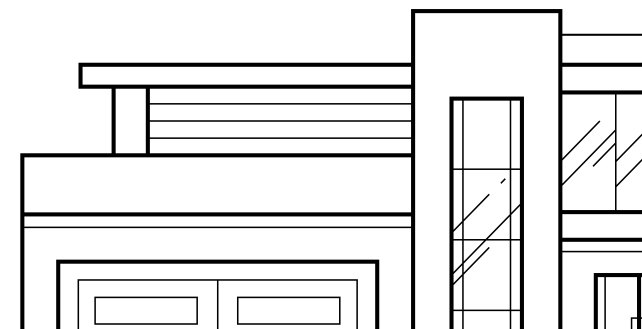


1. Policymakers (DoB, City of New York, FDNY, etc)

2. Renters/buyers/sellers

3. Housing agents

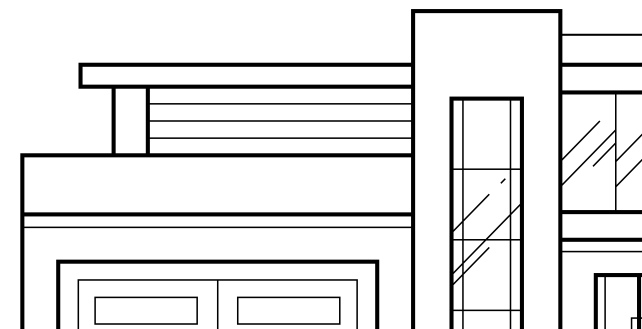
4. Our Mayor?



Limitations



1. Using Census data for to better understand housing affordability (e.g. in Bronx)
2. Still looking for sub-borough GEO JSON!
3. Missing data - 2025 new building applications.



Thank You

- **Streamlit:** <https://cosmic-spaghetti.streamlit.app/>
- **GitHub Repo:** <https://github.com/advanced-computing/cosmic-spaghetti>

